

Introduction - VueJS

Dr. B. Prabavathy
Associate Professor



Full-Stack

- Development approach that **combines the frontend, backend, and database into a complete application.**
-

Full-Stack Technology Stack

- **Vue.js (requires node runtime environment) + Spring Boot + MongoDB**
- **Vue.js**
 - Frontend framework used to build interactive and responsive user interfaces.
- **Spring Boot**
 - Backend framework used to develop REST APIs and implement business logic.
- **MongoDB**
 - NoSQL database used to store and manage application data.
- Ryan Dahl - created **Node run time environment** in 2009. He took Google's V8 JavaScript engine (used by Chrome) and combined it with an event-driven, non-blocking I/O system to allow JavaScript to run outside the browser.
- For a Vue.js application, npm and Vite have different roles:
 - npm → Manages the packages/dependencies needed by the Vue.js project, such as Vue itself.
 - Vite → Provides the development server and build tool that runs your Vue application during development and prepares it for production.

What is VueJS?

- JavaScript framework for building user interfaces, commonly used for [Single-Page Applications \(SPAs\)](#)
- SPA (Single-Page Application) is a web application that loads a single page and dynamically updates its content without reloading the entire page.
- Created by [Evan You](#) and maintained by the Vue.js core team and open-source community.

Key Features of VueJS

- Progressive & Flexible Framework
 - Vue can be used gradually—from adding a small interactive feature to an existing HTML page to developing a complete Single Page Application (SPA).
- Components:
 - They help in **creating custom elements** that can be reused in HTML.
- Component-based Architecture
 - Vue applications are divided into small, reusable components, making the application easier to develop and maintain.
- Reactive Data Binding and Virtual DOM
 - Vue **automatically updates the user interface** when data changes and uses the Virtual DOM to update only the necessary parts efficiently.
- Directives
 - Vue provides special directives such as **v-if, v-for, v-model, and v-bind** to control HTML elements and handle dynamic content.

A Sample Vue App

Editor

VS code

Check the versions of node and npm

node – version 22

npm - version 8

How to create VueApp?

npm create vue@latest my-vue-app1

cd my-vue-app1

npm install

npm run dev

```
✓ MY-VUE-APP1
  > .vscode
  > node_modules
  > public
  > src
  .gitignore
  index.html
  {} package.json
  {} package-lock.json
  README.md
  vite.config.js
  jsconfig.json
```

```
✓ MY-VUE-APP1
  > .vscode
  > node_modules
  > public
  ✓ src
    > assets
    > components
    ✓ App.vue
    JS main.js
    .gitignore
    index.html
    {} package.json
    {} package-lock.json
    README.md
    vite.config.js
    jsconfig.json
```

Exploring the Project Structure

- `node_modules/`: Contains **all the dependencies** and sub-dependencies of your project.
- `public/`: Contains the public assets of your application, including the `index.html` file which is the entry point of your application.
- `src/`: Contains the source code of your application. This is where you'll spend most of your time.
 - `App.test.js`: The test file for your main application component.
 - `main.js`: The JavaScript entry point of your application.
 - `App.vue` – Root component of the application
- `package.json`: Contains **metadata** about your project and its dependencies.

package.json

```
{  
  "name": "my-vue-app1",  
  "version": "0.0.0",  
  "private": true,  
  "type": "module",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "preview": "vite preview"  
  },  
  "dependencies": {  
    "vue": "^3.5.40"  
  },  
  "devDependencies": {  
    "@vitejs/plugin-vue": "^6.0.8",  
    "vite": "^8.1.5",  
    "vite-plugin-vue-devtools": "^8.1.5"  
  },  
  "engines": {  
    "node": "^22.18.0 || >=24.12.0"  
  }  
}
```

index.html

```
<!DOCTYPE html>  
<html lang="">  
  <head>  
    <meta charset="UTF-8">  
    <link rel="icon" href="/favicon.ico">  
    <meta name="viewport" content="width=device-width, initial-  
scale=1.0">  
    <title>Vite App</title>  
  </head>  
  <body>  
    <div id="app"></div>  
    <script type="module" src="/src/main.js"></script>  
  </body>  
</html>
```

main.js

```
import './assets/main.css'
```

→ imports the global CSS file

```
import { createApp } from 'vue'
```

→ imports Vue's createApp() function to create a Vue application.

```
import App from './App.vue'
```

→ Imports the **root component** (App.vue) of the Vue application

```
createApp(App).mount('#app')
```

→ creates the Vue application using App.vue and mounts it to the HTML element with the **ID app**, thereby displaying the Vue application in the browser.

A Vue component generally has **three main parts**:

<script setup>

const name = "Prabavathy"

</script>

<template>

<h1>Hello, {{ name }}</h1>

</template>

<style scoped>

h1 {

color: blue;

}

</style>

Structure

<template> → Defines what appears on the webpage (UI).

<script setup> → Contains JavaScript logic, data, functions, and component behavior.

<style> → Defines the CSS styling of the component.

So, simply:

Vue Component = Template (UI) + Script (Logic) + Style (Design).

<script setup> = "The JavaScript code inside this component is automatically treated as the component's setup logic."

scoped in **<style scoped>** means that the CSS styles will apply only to that particular component, rather than affecting elements in other components.

const → value should not be reassigned

let → value may be reassigned

var → older way of declaring variables; generally avoid it in modern JavaScript.

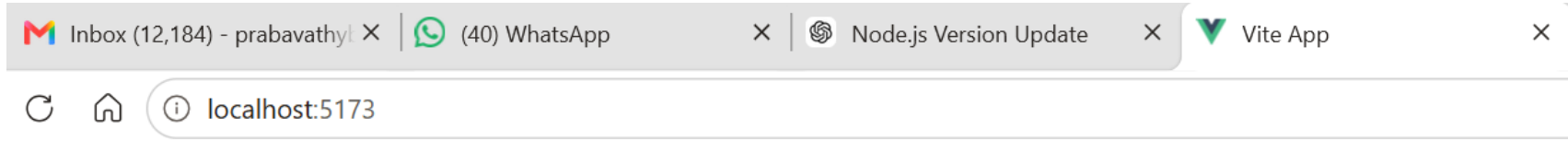
Ex1: HelloWorld – Vue Application

HelloWorld.vue

```
<template>
  <h1>Hello World Praba </h1>
</template>
<script setup>
</script>
```

App.vue

```
<script setup>
import HelloWorld from './components/HelloWorld.vue'
</script>
<template>
  <HelloWorld/>
</template>
<script setup>
</script>
```



Hello World Praba